

```

import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader
import matplotlib.pyplot as plt
import numpy as np

transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])

train_data = datasets.CIFAR10(root='./data', train=True, download=True, transform=transform)
test_data = datasets.CIFAR10(root='./data', train=False, download=True, transform=transform)

train_loader = DataLoader(train_data, batch_size=128, shuffle=True)
test_loader = DataLoader(test_data, batch_size=8, shuffle=True)

import torch
import torch.nn as nn

class VisionTransformer(nn.Module):
    def __init__(self, patch_size=4, num_classes=10, dim=128, depth=2, heads=4, mlp_dim=256):
        super().__init__()
        num_patches = (32 // patch_size) ** 2 # CIFAR-10 images are 32x32
        self.patch_embed = nn.Conv2d(3, dim, kernel_size=patch_size, stride=patch_size)

        self.cls_token = nn.Parameter(torch.randn(1, 1, dim))
        self.pos_embedding = nn.Parameter(torch.randn(1, num_patches + 1, dim))
        self.dropout = nn.Dropout(0.1)

        self.transformer = nn.Sequential(*[
            nn.TransformerEncoderLayer(d_model=dim, nhead=heads, dim_feedforward=mlp_dim, batch_first=True)
            for _ in range(depth)
        ])

        self.norm = nn.LayerNorm(dim)
        self.head = nn.Linear(dim, num_classes)

    def forward(self, x):
        # x shape: [B, 3, 32, 32]
        x = self.patch_embed(x) # shape: [B, dim, 8, 8] if patch_size=4
        x = x.flatten(2).transpose(1, 2) # shape: [B, N, dim] where N = 8x8 = 64

        B, N, D = x.shape
        cls_token = self.cls_token.expand(B, -1, -1) # [B, 1, dim]
        x = torch.cat((cls_token, x), dim=1) # [B, N+1, dim]
        x = self.dropout(x + self.pos_embedding[:, :N+1, :]) # add positional embeddings

        x = self.transformer(x)
        x = self.norm(x[:, 0]) # use only the [CLS] token
        return self.head(x)

def train(model, loader, optimizer, criterion, device):
    model.train()
    total_loss = 0
    for imgs, labels in loader:
        imgs, labels = imgs.to(device), labels.to(device)
        optimizer.zero_grad()
        outputs = model(imgs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        total_loss += loss.item()
    return total_loss / len(loader)

def test(model, loader, device):
    model.eval()
    correct, total = 0, 0
    with torch.no_grad():
        for imgs, labels in loader:
            imgs, labels = imgs.to(device), labels.to(device)
            outputs = model(imgs)
            _, preds = torch.max(outputs, 1)
            correct += (preds == labels).sum().item()
            total += labels.size(0)

```

```

return 100 * correct / total

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = VisionTransformer().to(device)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=1e-3)

for epoch in range(10):
    loss = train(model, train_loader, optimizer, criterion, device)
    acc = test(model, test_loader, device)
    print(f"Epoch {epoch+1}: Loss={loss:.4f}, Accuracy={acc:.2f}%")

```

```

↩ Epoch 1: Loss=1.3574, Accuracy=54.37%
Epoch 2: Loss=1.2350, Accuracy=57.52%
Epoch 3: Loss=1.1915, Accuracy=59.04%
Epoch 4: Loss=1.1453, Accuracy=60.70%
Epoch 5: Loss=1.1147, Accuracy=61.46%
Epoch 6: Loss=1.0890, Accuracy=61.24%
Epoch 7: Loss=1.0701, Accuracy=62.02%
Epoch 8: Loss=1.0511, Accuracy=62.06%
Epoch 9: Loss=1.0287, Accuracy=62.91%
Epoch 10: Loss=1.0205, Accuracy=63.14%

```

```

def fgsm_attack(image, epsilon, gradient):
    sign_data_grad = gradient.sign()
    perturbed_image = image + epsilon * sign_data_grad
    perturbed_image = torch.clamp(perturbed_image, -1, 1)
    return perturbed_image

```

```

classes = train_data.classes

```

```

def imshow(img, title):
    img = img / 2 + 0.5
    npimg = img.cpu().numpy()
    plt.imshow(np.transpose(npimg, (1, 2, 0)))
    plt.title(title)
    plt.axis('off')

```

```

images, labels = next(iter(test_loader))
images, labels = images.to(device), labels.to(device)
outputs = model(images)
_, preds = torch.max(outputs, 1)

```

```

plt.figure(figsize=(12, 4))
for i in range(8):
    plt.subplot(2, 4, i+1)
    imshow(images[i], f"Pred: {classes[preds[i]]}")
plt.tight_layout()
plt.show()

```



```

for i in range(len(images)):
    pred_label = classes[preds[i]]
    true_label = classes[labels[i]]

```

```

title = f"True: {true_label}\nPred: {pred_label}"
plt.subplot(2, 4, i+1)
imshow(images[i], title)

```



```

for i in range(len(images)):
    is_correct = preds[i] == labels[i]
    color = 'green' if is_correct else 'red'
    title = f"{classes[preds[i]]} ({'✓' if is_correct else '✗'})"
    plt.subplot(2, 4, i+1)
    imshow(images[i], title)
    plt.gca().title.set_color(color)

```



```

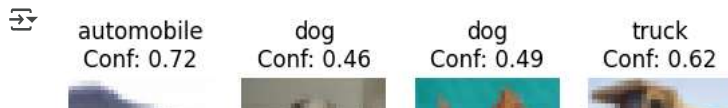
softmax = nn.Softmax(dim=1)
probs = softmax(outputs)

```

```

for i in range(len(images)):
    pred_label = classes[preds[i]]
    confidence = probs[i][preds[i]].item()
    title = f"{pred_label}\nConf: {confidence:.2f}"
    plt.subplot(2, 4, i+1)
    imshow(images[i], title)

```



```
def fgsm_attack(image, epsilon, data_grad):
    sign_data_grad = data_grad.sign()
    perturbed_image = image + epsilon * sign_data_grad
    perturbed_image = torch.clamp(perturbed_image, -1, 1) # keep in valid range
    return perturbed_image
```

```
dog          horse          dog          automobile
epsilon = 0.1 # perturbation strength
```

```
def train_adversarial(model, loader, optimizer, criterion, device, epsilon):
    model.train()
    total_loss = 0
    for imgs, labels in loader:
        imgs, labels = imgs.to(device), labels.to(device)
        imgs.requires_grad = True

        # Forward on clean image
        outputs = model(imgs)
        loss = criterion(outputs, labels)
        model.zero_grad()
        loss.backward()

        # Generate adversarial example
        imgs_adv = fgsm_attack(imgs, epsilon, imgs.grad.data)

        # Forward on adversarial image
        outputs_adv = model(imgs_adv.detach())
        loss_adv = criterion(outputs_adv, labels)

        optimizer.zero_grad()
        loss_adv.backward()
        optimizer.step()
        total_loss += loss_adv.item()

    return total_loss / len(loader)

for epoch in range(5):
    loss = train_adversarial(model, train_loader, optimizer, criterion, device, epsilon)
    acc = test(model, test_loader, device)
    print(f"Epoch {epoch+1}: Adv Loss={loss:.4f}, Accuracy={acc:.2f}%")
```

```
Epoch 1: Adv Loss=2.2275, Accuracy=34.19%
Epoch 2: Adv Loss=2.1087, Accuracy=41.62%
Epoch 3: Adv Loss=2.0762, Accuracy=40.19%
Epoch 4: Adv Loss=2.0552, Accuracy=39.81%
Epoch 5: Adv Loss=2.0396, Accuracy=40.92%
```

Start coding or [generate](#) with AI.